

Trabalho Prático 2

Projetando um Gerenciador de Memória Virtual

Relatório Técnico

PUC Minas — Sistemas Operacionais

1. Introdução

Este relatório descreve o desenvolvimento do Trabalho Prático 2, cujo objetivo é implementar, em linguagem C, um simulador de gerenciamento de memória virtual baseado em paginação. O programa traduz endereços lógicos de 16 bits para endereços físicos, utilizando uma Tabela de Páginas com 256 entradas e uma TLB (Translation Lookaside Buffer) com 16 entradas.

O cenário simulado segue o modelo clássico de tradução de endereços apresentado em sistemas operacionais: o espaço de endereçamento virtual possui 65.536 bytes (2^{16}), organizado em 256 páginas de 256 bytes cada. A memória física, por sua vez, é menor (32.768 bytes, ou 128 quadros de 256 bytes), o que torna necessário o uso de paginação por demanda e de uma política de substituição de páginas quando não há quadros livres disponíveis.

Como fundamentação teórica, o trabalho aborda três conceitos centrais de gerência de memória: (1) tradução de endereços por paginação, em que o endereço lógico é dividido em número da página e deslocamento (offset); (2) o uso de uma TLB como cache de tradução para acelerar o acesso, reduzindo a necessidade de consultar a tabela de páginas a cada referência; e (3) o algoritmo de Aging, uma aproximação eficiente do algoritmo LRU (Least Recently Used) para escolha de páginas vítimas, que substitui o registro de timestamps por um contador de envelhecimento de 8 bits atualizado periodicamente a partir do bit de referência de cada página.

2. Arquitetura e Estruturas de Dados

O projeto manteve a organização modular fornecida no projeto-base, separando responsabilidades em cinco módulos:

- `main.c` — laço principal de leitura dos endereços lógicos, extração de página/offset, orquestração entre TLB, tabela de páginas e memória física, e impressão dos resultados.
- `tlb.c` / `tlb.h` — TLB com 16 entradas e política de substituição FIFO.
- `page_table.c` / `page_table.h` — tabela de páginas com 256 entradas, cada uma contendo número do quadro, bit de validade, bit de referência e contador de aging de 8 bits.
- `memory.c` / `memory.h` — memória física (128 quadros de 256 bytes), tratamento de faltas de página, leitura do `BACKING_STORE.bin` e seleção de página vítima.
- `statistics.c` / `statistics.h` — contadores de endereços traduzidos, page faults e TLB hits, além do cálculo das taxas finais.

A principal estrutura de dados é a entrada da tabela de páginas:

```
typedef struct {    int frame;    int valid;    unsigned char
reference_bit;    unsigned char aging_counter; } page_table_entry_t;
```

O campo `aging_counter` é o elemento central da política de substituição: a cada acesso à memória, o contador de todas as páginas válidas é deslocado um bit para a direita e o bit de referência é inserido na posição mais significativa, sendo então zerado. Dessa forma, páginas acessadas recentemente mantêm contadores com bits altos ligados, enquanto páginas não acessadas tendem a zerar o contador ao longo do tempo — exatamente o comportamento esperado do algoritmo de Aging.

A TLB foi modelada como um vetor de 16 entradas (`page`, `frame`, `valid`), com um índice `fifo_next` que indica a próxima posição a ser sobrescrita quando todas as entradas estiverem ocupadas, implementando assim a política FIFO exigida pela especificação.

3. Implementação e Decisões de Projeto

3.1 Extração de página e offset

Como o arquivo de entrada contém inteiros de 32 bits, o primeiro passo é mascarar o endereço lógico com `0xFFFF` para manter apenas os 16 bits menos significativos. Em seguida, o número da página é obtido deslocando o endereço 8 bits para a direita e aplicando a máscara `0xFF`, e o offset é obtido aplicando `0xFF` diretamente sobre o endereço mascarado:

```
logical_address = logical_address & 0xFFFF; int page    = (logical_address >>
8) & 0xFF; int offset = logical_address & 0xFF;
```

3.2 Fluxo de tradução

Para cada endereço lido, o programa primeiro consulta a TLB. Em caso de TLB hit, o quadro é obtido diretamente, evitando a consulta à tabela de páginas. Em caso de TLB miss, a tabela de páginas é consultada; se a entrada for válida, o quadro é obtido dali e inserido na TLB; caso contrário, ocorre uma falta de página (`page fault`), tratada pela função `handle_page_fault`.

3.3 Tratamento de faltas de página

A função `handle_page_fault` primeiro procura um quadro livre. Se não houver quadro disponível, a página vítima é escolhida via `select_victim_page` (menor `aging_counter` entre as páginas válidas), sua entrada é invalidada na tabela de páginas e removida da TLB, e o quadro correspondente é liberado para reutilização. Em seguida, a página solicitada é lida do arquivo `BACKING_STORE.bin` usando `fseek` (posicionando-se em `page * PAGE_SIZE`) e `fread` (lendo `PAGE_SIZE` bytes), o quadro é atualizado em `frame_to_page` e a tabela de páginas é atualizada com o novo mapeamento.

3.4 Política de substituição (LRU aproximado / Aging)

A cada acesso, `page_table_set_reference` marca o bit de referência da página acessada, e `page_table_update_aging` desloca o contador de aging de todas as páginas válidas, inserindo o bit de referência no bit mais significativo e zerando-o em seguida. Quando uma substituição é necessária, `select_victim_page` percorre as páginas válidas e escolhe aquela com o menor valor de `aging_counter`, aproximando o comportamento do algoritmo LRU sem a necessidade de armazenar históricos completos de acesso.

3.5 TLB com FIFO

A inserção na TLB primeiro verifica se a página já está presente (atualizando o quadro, se for o caso); em seguida procura por uma entrada inválida livre; e, somente se a TLB estiver completamente ocupada, substitui a entrada apontada por `fifo_next`, avançando esse índice de forma circular — caracterizando a política FIFO exigida.

3.6 Estatísticas finais

Os contadores `total_addresses`, `page_faults` e `tlb_hits` são incrementados ao longo da execução, e as taxas finais são calculadas como percentuais simples ($\text{contagem} / \text{total} * 100$), impressas ao final da execução juntamente com os valores absolutos.

3.7 Dificuldades encontradas

- Garantir a ordem correta de invalidação da página vítima (tabela de páginas e TLB) antes de reaproveitar o quadro, evitando inconsistências entre as duas estruturas.
- Tratar corretamente o tipo `signed char` na leitura da memória física, já que valores de byte acima de 127 aparecem como números negativos na saída — comportamento esperado e documentado no script `generate_data.py`.
- Posicionar corretamente o cursor do arquivo `BACKING_STORE.bin` com `fseek`, multiplicando o número da página por `PAGE_SIZE`, para não desalinhar a leitura das páginas.

4. Resultados Obtidos e Análise

O simulador foi executado com os dois arquivos de teste gerados por `generate_data.py` (10.000 endereços cada), com seed fixa (42) para reprodutibilidade. Os resultados obtidos foram:

Métrica	<code>addresses_random.txt</code>	<code>addresses_location.txt</code>
Endereços traduzidos	10.000	10.000
Page Faults	4.958	1.164
Taxa de Page Faults	49,58%	11,64%
TLB Hits	654	7.925
Taxa de TLB Hit	6,54%	79,25%

A análise dos resultados confirma o comportamento esperado em relação à localidade de referência. No arquivo `addresses_random.txt`, os endereços são gerados de forma totalmente aleatória dentro do espaço de 65.536 bytes; como não há reuso intencional de páginas próximas, tanto a taxa de TLB hit quanto o aproveitamento das páginas já carregadas em memória são baixos, resultando em quase 50% de page faults.

Já no arquivo `addresses_location.txt`, 85% dos acessos ocorrem em páginas próximas à página corrente (delta de até 2 páginas), simulando localidade espacial e temporal típica de programas reais. Esse padrão favorece fortemente tanto a TLB (79,25% de acertos) quanto a tabela de páginas, reduzindo a taxa de page faults para 11,64%. Esse contraste evidencia, na prática,

por que sistemas reais de memória virtual se beneficiam tanto da localidade de referência observada em cargas de trabalho típicas.

A correção da tradução de endereços foi validada manualmente: como o BACKING_STORE.bin é gerado de forma determinística ($\text{byte } i = i \bmod 256$), o valor lido para cada endereço lógico deve corresponder a ($\text{endereço lógico} \bmod 256$), interpretado como signed char. Por exemplo, o endereço lógico 3278 resulta em $3278 \bmod 256 = 206$, que como signed char (com wraparound) corresponde a -50 — valor exatamente reproduzido pela execução do simulador, confirmando a correção da extração de página/offset, do cálculo do endereço físico e da leitura do BACKING_STORE.

O programa foi compilado com `gcc -Wall -Wextra -O2 -std=c11`, sem nenhum warning, e executado com sucesso para ambos os arquivos de entrada.

5. Conclusão

O trabalho permitiu consolidar, na prática, os conceitos de tradução de endereços por paginação, uso de TLB como mecanismo de cache de tradução e políticas de substituição de páginas baseadas em aproximações do LRU. A implementação do algoritmo de Aging mostrou-se uma solução simples e eficiente para aproximar o comportamento do LRU sem o custo de manter timestamps completos, e os resultados obtidos com os dois padrões de acesso (aleatório e com localidade) evidenciaram de forma clara o impacto da localidade de referência sobre o desempenho do TLB e sobre a taxa de page faults. A organização modular do projeto-base também facilitou o desenvolvimento incremental sugerido no enunciado, permitindo validar cada componente (tabela de páginas, memória física, page faults, substituição de páginas e TLB) de forma isolada antes da integração final.

6. Uso de Inteligência Artificial (IA)

Conforme exigido pela Seção 9 do enunciado, esta seção documenta de forma transparente o uso de ferramentas de IA (assistente Claude, da Anthropic) durante o desenvolvimento, seguindo a metodologia Spec-Driven Development (SDD). Os prompts foram elaborados de forma a definir claramente o problema, especificar requisitos funcionais e não funcionais, e indicar restrições de implementação, evitando solicitações vagas.

6.1 Prompt 1 — Especificação do módulo de tradução de endereços

Prompt utilizado:

```
"Implemente, em C11, a extração do número de página e do offset a partir de um endereço lógico de 32 bits, considerando: (1) apenas os 16 bits menos significativos são válidos; (2) os 8 bits mais altos desses 16 formam o número da página (0-255); (3) os 8 bits mais baixos formam o offset (0-255); (4) o cálculo do endereço físico deve ser frame*FRAME_SIZE + offset, com FRAME_SIZE=256. Não altere as assinaturas das funções já definidas em main.c/config.h."
```

Resposta/uso: a IA sugeriu o uso de operadores de máscara e deslocamento de bits (`& 0xFFFF`, `>> 8`, `& 0xFF`), explicando a relação entre o tamanho da página ($256 = 2^8$) e o número de bits do offset. O código gerado foi revisado e testado manualmente, confirmando a correta separação de página e offset para múltiplos endereços de teste.

6.2 Prompt 2 — Tratamento de faltas de página e substituição com Aging

Prompt utilizado:

"Especifique e implemente, em C11, a função `handle_page_fault(int page)` do módulo `memory.c`, com os seguintes requisitos funcionais: (1) procurar um quadro livre; (2) se não houver, selecionar uma página vítima usando o algoritmo de Aging (menor `aging_counter` entre as páginas válidas); (3) invalidar a página vítima na tabela de páginas e remover sua entrada da TLB; (4) ler a página solicitada do arquivo `BACKING_STORE.bin` usando `fseek/fread`, posicionando o cursor em `page*PAGE_SIZE`; (5) atualizar a tabela de páginas e o mapeamento quadro->página; (6) retornar o número do quadro. Restrição: usar apenas funções padrão da biblioteca C (`stdio.h`), sem bibliotecas externas."

Resposta/uso: a IA propôs a ordem de operações (verificar quadro livre → selecionar vítima → invalidar estruturas → ler do backing store → atualizar tabela), destacando a importância de invalidar a TLB antes de reaproveitar o quadro, para evitar que uma entrada antiga da TLB aponte para um quadro que já foi reatribuído a outra página. O código foi adaptado às assinaturas do projeto-base e testado com os dois arquivos de endereços fornecidos.

6.3 Prompt 3 — Atualização dos contadores de aging (LRU aproximado)

Prompt utilizado:

"Explique e implemente o algoritmo de Aging para LRU aproximado, em C11, usando um contador de 8 bits por página (`aging_counter`) e um bit de referência (`reference_bit`), com a seguinte regra de atualização periódica, aplicada a todas as páginas válidas: (1) deslocar `aging_counter` um bit para a direita; (2) inserir `reference_bit` no bit mais significativo de `aging_counter`; (3) zerar `reference_bit`. A página vítima deve ser aquela com o menor valor de `aging_counter` entre as páginas válidas."

Resposta/uso: a IA explicou que esse esquema aproxima o LRU porque páginas referenciadas recentemente acumulam bits 1 nas posições mais significativas do contador, enquanto páginas não referenciadas tendem a zerar o contador com o passar do tempo, sem necessidade de armazenar uma lista ordenada de acessos. A explicação foi usada para validar a implementação de `page_table_update_aging` e `select_victim_page`, e para justificar a escolha de projeto na Seção 3.4 deste relatório.

6.4 Prompt 4 — Geração da seção de testes e validação

Prompt utilizado:

"Dado que o `BACKING_STORE.bin` é gerado de forma determinística (`byte i = i mod 256`) e os arquivos de endereços `addresses_random.txt` e `addresses_location.txt` são gerados com seed fixa, sugira uma forma simples de validar manualmente a corretude da tradução de endereços e da leitura de memória, sem depender de um gabarito externo."

Resposta/uso: a IA sugeriu comparar o valor de saída de cada endereço lógico com (endereço lógico mod 256), interpretado como signed char, já que essa é a regra de geração do backing store. Essa verificação foi aplicada manualmente a algumas linhas da execução real do programa (Seção 4) e confirmou a corretude da tradução implementada.

6.5 Responsabilidade técnica

Todo o código gerado com auxílio de IA foi revisado, compilado, executado e testado pelos integrantes do grupo antes da entrega. O grupo é integralmente responsável pelo conteúdo entregue e está apto a explicar e defender cada decisão de implementação descrita neste relatório, conforme exigido na Seção 9.3 do enunciado.

Repositório GitHub: [inserir aqui o link do repositório público do grupo]

Integrantes do grupo: [inserir nomes e matrículas dos integrantes]